Ai programs used:

Claude vs. Gemini

(Note: Claude is set for student use purpose where Gemini isn't)

How I accessed them:

Gemini…gemini.google.com on the browser

Claude…claude.ai on windows

Prompt used:

Objective

Write a C program that picks a secret number between 1 and 10.

The user gets up to 3 tries to guess the number.

Requirements

• Use a fixed secret number (for example, 7).

• Use printf to prompt the user and scanf to read their guess.

• Use a while or for loop to let the user guess up to 3 times.

• Use if-else statements to check if the guess is too high, too low, or correct.

• Stop the loop immediately if the user guesses the correct number.

• Print a final win or lose message.

Example Output

text

Guess a number between 1 and 10.

Attempt 1/3. Enter your guess: 5

Too low! Try again.

Attempt 2/3. Enter your guess: 8

Too high! Try again.

Attempt 3/3. Enter your guess: 7

Correct! You win!

Generated Code:

Claude

```
#include <stdio.h>

int main() {
    int secret = 7;
    int guess;
    int attempts = 3;
    int won = 0;
```

```
    printf("Guess a number between 1 and 10.\n");

    for (int i = 1; i <= attempts; i++) {
        printf("Attempt %d/%d. Enter your guess: ", i, attempts);
        scanf("%d", &guess);

        if (guess == secret) {
            printf("Correct! You win!\n");
            won = 1;
            break;
        } else if (guess < secret) {
            printf("Too low! Try again.\n");
        } else {
            printf("Too high! Try again.\n");
        }
    }

    if (!won) {
        printf("Sorry, you lose! The number was %d.\n", secret);
    }

    return 0;
}
```

Gemini

```
#include <stdio.t>
#include <stdio.h>

int main() {
    const int SECRET_NUMBER = 7;
    const int MAX_ATTEMPTS = 3;
    int guess;
    int guessed_correctly = 0;

    printf("Guess a number between 1 and 10.\n");

    for (int attempt = 1; attempt <= MAX_ATTEMPTS; attempt++) {
        printf("Attempt %d/%d. Enter your guess: ", attempt, MAX_ATTEMPTS);
        scanf("%d", &guess);

        if (guess == SECRET_NUMBER) {
            printf("Correct! You win!\n");
            guessed_correctly = 1;
```

```
            break; // Stop the loop immediately on a correct guess
        } else if (guess < SECRET_NUMBER) {
            if (attempt < MAX_ATTEMPTS) {
                printf("Too low! Try again.\n");
            }
        } else {
            if (attempt < MAX_ATTEMPTS) {
                printf("Too high! Try again.\n");
            }
        }
    }

    if (!guessed_correctly) {
        printf("Out of attempts! You lose. The secret number was %d.\n", SECRET_NUMBER);
    }

    return 0;
}
```

Analysis:

1. Correctness
   For the first pass, Gemini doesn't run at all since it has <stdio.t> typo. Claude does run the program almost as expected except that when you run out of tries, it still outputs the message "try again!"
   Where if Geminis program did run, it will not.

2. Execution Time
   Close to no difference, but since Gemini uses more if statements, it will run ever so slightly slower.

3. Space Complexity
   Identical, both use the same amount of variables.

4. Maintainability
   Gemini has a single line of comment that explains the line of code where Claude doesn't. However, most importantly gemini uses the "const" keyword for the secret number and max attempt variable and uses all caps for the names as well, making it clear for the person reading that it is a value that shouldn't change. Claude doesn't have any of those maintainability precautions.

Selecting a base:

Even though Gemini has a critical error, it has better practice for code maintainability and the output is preferable than Claude. So I will be using Gemini's code for the base.

Edits:

1. Correctness
   Take out the <stdio.t> typo.
   Add in error checking for invalid inputs (like characters that are not numbers and ints that are out of range)

2. Space complexity
   It is about as optimal as it gets and there are no meaningful fixes we can do.

3. Execution time
   It is about as optimal as it gets and there are no meaningful fixes we can do.

4. Maintainability
   Add in more comments to explain what each part of the code does.